

Gradient Boosting C++-ban

Megfigyelt párok (x, y) , ismeretlen $f : \mathcal{X} \rightarrow \mathcal{Y}$, ahol $\mathcal{X} \subseteq \mathbb{R}^p$, $\mathcal{Y} \subseteq \mathbb{R}$.

Keressünk $\hat{f} : \mathcal{X} \rightarrow \mathcal{Y}$ -t véges mintából:

$$D = \{(x_1, y_1), \dots, (x_N, y_N)\} \subset \mathcal{X} \times \mathcal{Y}$$

A gradient boosting $\hat{f}$ -et sok kis *gyenge tanuló* (sekély fa) összegeként építi fel, mindegyik az előzők hibáját korrigálja.

$X \in \mathbb{R}^{N \times p}$ mátrix (sor = megfigyelés, oszlop = feature), $Y \in \mathbb{R}^N$ célvektor.

Sor-főrendű tárolás: $\text{idx}(i, j) = i \cdot \text{cols} + j$

```
struct Matrix {  
    size_t rows, cols;  
    std::vector<double> data;  
    double& at(size_t i, size_t j) { return data[i*cols + j]; }  
};
```

Veszteségfüggvény $\mathcal{L} : \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}_{\geq 0}$. Választjuk $\hat{f}$ -et az átlagos veszteség minimalizálásával:

$$\hat{f} = \arg \min_f \frac{1}{N} \sum_{i=1}^N \mathcal{L}(y_i, f(x_i))$$

L_2 veszteség (MSE) folytonos regresszióhoz:

$$\mathcal{L}(y, \hat{y}) = \frac{1}{2}(y - \hat{y})^2$$

Regressziós fa: $\mathcal{X}$ -et J diszjunkt régióra particionálja, régióként konstans jósol.

$$\hat{f}(x) = \sum_{j=1}^J c_j \mathbb{1}(x \in R_j)$$

Indexhalmaz régióként: $I_j = \{i : x_i \in R_j\}$

Minimalizálandó össz-SSE:

$$\text{SSE} = \sum_{j=1}^J \sum_{i \in R_j} (y_i - c_j)^2$$

Fixált R_j -re c_j szerint deriválva:

$$-2 \sum_{i \in R_j} (y_i - c_j) = 0 \quad \Rightarrow \quad \hat{c}_j = \frac{1}{|I_j|} \sum_{i \in R_j} y_i$$

A levél optimuma az ott lévő célértékek számtani közepe.

Globális optimum NP-nehéz. Csúcsenként a legjobb (k, s) párt keressük:

$$R_1(k, s) = \{x \in R : x_k \leq s\}, \quad R_2(k, s) = \{x \in R : x_k > s\}$$

$$\min_{k,s} \left[\sum_{i \in I_1(k,s)} (y_i - \hat{c}_1)^2 + \sum_{i \in I_2(k,s)} (y_i - \hat{c}_2)^2 \right]$$

Ekvivalens a variancia-csökkenés $\Delta(k, s)$ maximalizálásával.

Kulcs azonosság — inkrementális momentum frissítés:

$$\sum_{i \in R} (y_i - \bar{y})^2 = \sum_{i \in R} y_i^2 - \frac{1}{|R|} \left(\sum_{i \in R} y_i \right)^2$$

```
sum_left += t; sum_sq_left += t*t; n_left++;  
sum_right -= t; sum_sq_right -= t*t; n_right--;  
  
sse = (sum_sq_left - sum_left*sum_left/n_left)  
      + (sum_sq_right - sum_right*sum_right/n_right);
```

Teljes keresés p feature felett: $O(pN \log N)$ csúcsenként.

```
struct Node {  
    int    feature_idx = -1; // k  (-1 = levél)  
    double threshold   = 0.0; // s  
    double prediction   = 0.0; //  $\hat{c}_j$   
    int    left_child  = -1; // vektor index  
    int    right_child = -1;  
};
```

Pointer-mentes `std::vector<Node>`, gyerekek vektor-indexszel. Megállási feltételek: max. mélység, kevesebb mint 2 minta, vagy nincs érvényes osztás.

Nagyon mély fa tökéletesen illeszkedik tanító adathalmazra — magas variancia.
Egyetlen adatpont megváltozása megváltoztathatja a gyökér kettéosztási pontját.

Megoldás: sok sekély fa szekvenciálisan, összeadogatva

Paramétertérben (lineáris regresszió, neurális hálók): véges $\theta \in \mathbb{R}^d$.

$$\theta_m = \theta_{m-1} - \eta \nabla_{\theta} \mathcal{L}(\theta_{m-1})$$

Függvénytérben (boosting): nincs rögzített θ , maga a prediktáló F a változó.

Gradiens pontonként:

$$g_i = \frac{\partial \mathcal{L}(y_i, F(x_i))}{\partial F(x_i)} \Big|_{F=F_{m-1}}$$

$$\frac{\partial \mathcal{L}(y_i, F(x_i))}{\partial F(x_i)} = -(y_i - F(x_i))$$

$$-g_i = y_i - F(x_i)$$

L_2 -nél a negatív gradiens a reziduum.

$$F_{m(x_i)} = F_{m-1}(x_i) + \eta(y_i - F_{m-1}(x_i))$$

$-g_i$ csak a megfigyelt N ponton van értelmezve. Sekély fa $h_{m(x)}$ illeszthető $-g$ -re, ez nem látott adatpontokra is kiértékelhetővé teszi:

$$F_{m(x)} = F_{m-1}(x) + \eta h_{m(x)}$$

Az együttes (ensemble): *gradient descent* egy végtelen dimenziós függvényterben, fákkal mint frissítési irányokkal.

Init: $F_0(x) = \bar{y}$. Aztán $m = 1, \dots, M$:

1. pszeudo-rezidum $r_{i,m} = y_i - F_{m-1}(x_i)$ kiszámolása
2. $h_{m(x)}$ sekély fa illesztése az (X, r_m) adatra
3. optimális levél-konstans $\gamma_{j,m}$ kiszámolása. ez L_2 -re automatikusan megtörténik a fa felépítése során.
4. frissítés: $F_{m(x)} = F_{m-1}(x) + \nu h_{m(x)}$

$\nu < 1$: tanulási ráta, regularizáció. Tipikus pl.: $\nu = 0.1$.

```
struct GradientBoostingRegressor {  
    int    n_estimators;  
    double learning_rate;  
    int    max_depth;  
    double initial_prediction;  
    std::vector<std::vector<Node>> ensemble;  
  
    void    fit(const Matrix& X, const std::vector<double>& Y);  
    double predict(const std::vector<double>& x_i) const;  
};
```

```
initial_prediction = mean(Y); // F_0
std::vector<double> F_m(N, initial_prediction);

for (int m = 0; m < n_estimators; ++m) {
    for (i = 0; i < N; ++i)
        pseudo_residuals[i] = Y[i] - F_m[i]; // 1.
    auto tree = build_tree(X, pseudo_residuals, ...); // 2+3.
    ensemble.push_back(tree);
    for (i = 0; i < N; ++i)
        F_m[i] += learning_rate * predict_single_tree(tree, x_i); // 4.
}
```

$$F_{M(x)} = F_0 + \nu \sum_{m=1}^M h_{m(x)}$$

```
double predict(const std::vector<double>& x_i) const {  
    double pred = initial_prediction;  
    for (const auto& tree : ensemble)  
        pred += learning_rate * predict_single_tree(tree, x_i);  
    return pred;  
}
```

Köszö a figyelmet + kérdések